# UPN, PASIÓN POR TRANSFORMAR VIDAS

## UNIDAD 1:

## LISTAS, PILAS Y COLAS

### SESIÓN 3:
### Listas enlazadas Dobles
### Listas enlazadas Circulares

Dr. Eric GustavoCoronel Castillo
eric.coronel@upn.pe

# MOTIVACIÓN

## DESAFIO DEL DIA

El verdadero éxito en la programación comienza con entender a fondo las estructuras que organizan y gestionan nuestros datos.

# MOTIVACIÓN

## IDEA FUERZA

Una base sólida en estructuras asegura programas robustos y mantenibles.

# LOGRO DE LA UNIDAD 1

Al finalizar la unidad, el estudiante implementa algoritmos utilizando listas simples, listas dobles, listas circulares, pilas y colas, a través de control de versiones y el lenguaje C# con entorno gráfico, demostrando lógica y habilidad en la implementación de los algoritmos.

# LOGRO DE LA SESIÓN

Al término de la sesión, el estudiante implementa algoritmos de listas doblemente enlazadas y circulares en C#, aplicando buenas prácticas de programación.

# REFLEXIONA

## PREGUNTA 1

**¿Cuál es la diferencia entre una lista simple y una doble?**

*💡 Piensa en la cantidad de punteros por nodo.*

## PREGUNTA 2

**¿Qué ventaja tiene poder recorrer la lista hacia atrás?**

*💡 Considera operaciones de eliminación y búsqueda inversa.*

## PREGUNTA 3

**¿Qué problema puede ocurrir en una lista circular?**

*💡 ¿Cómo sabe el algoritmo cuándo detenerse?*

# REFLEXIONA

## PREGUNTA 1

**¿Cuál es la diferencia entre una lista simple y una doble?**

La diferencia fundamental radica en la cantidad y dirección de los punteros por nodo. En una lista simplemente enlazada, cada nodo almacena un único puntero Next que apunta al nodo siguiente. En una lista doblemente enlazada, cada nodo mantiene dos punteros: Next (hacia adelante) y Prev (hacia atrás).

## PREGUNTA 2

**¿Qué ventaja tiene poder recorrer la lista hacia atrás?**

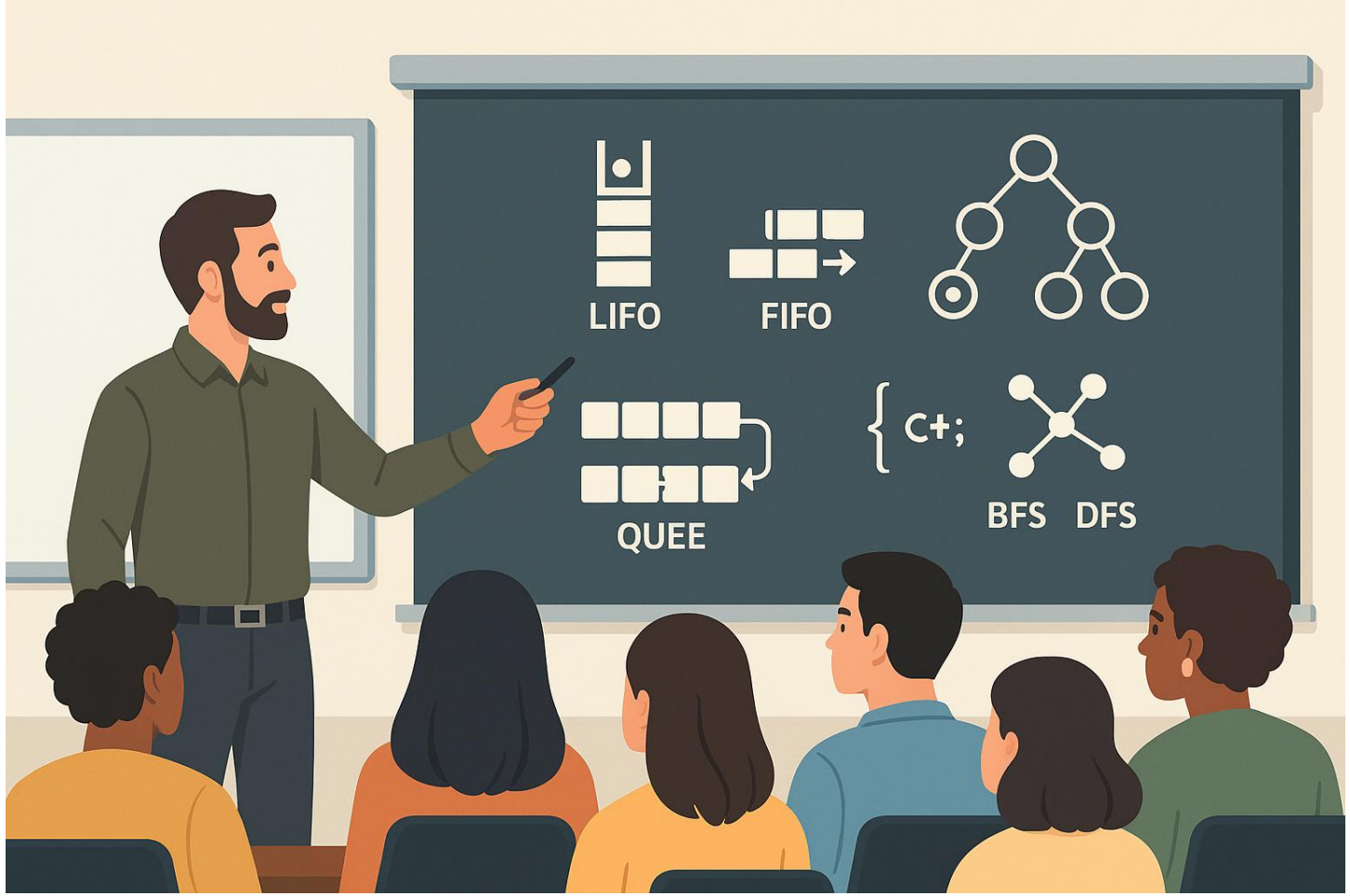
Las operaciones más beneficiadas son la eliminación en O(1) (sin necesidad de buscar el nodo previo) y la búsqueda inversa desde la cola.

## PREGUNTA 3

**¿Qué problema puede ocurrir en una lista circular?**

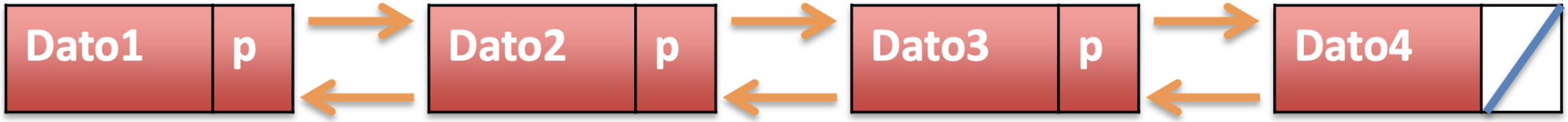
El problema principal es el bucle infinito (infinite loop). En una lista circular, el último nodo apunta de vuelta al primero, por lo que no existe un nodo nulo (null) que indique el fin de la lista. Si el algoritmo de recorrido verifica la condición de parada con `current != null`, nunca se detendrá y el programa se colgará.

# DESARROLLO



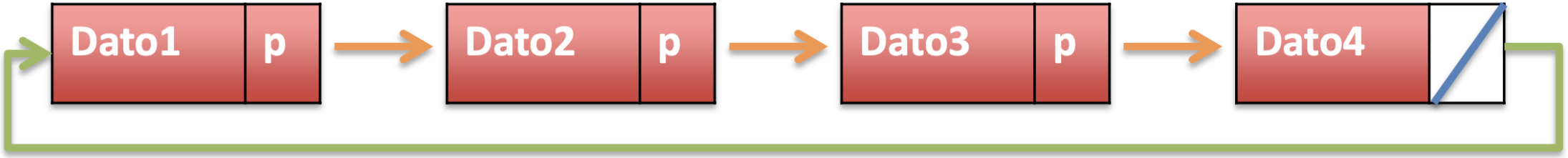
# LISTAS DOBLEMENTE ENLAZADAS

Cada nodo contiene dos enlaces, uno a su nodo predecesor y el otro a su nodo sucesor. La lista es eficiente tanto en recorrido directo («adelante») como en recorrido inverso («atrás»).



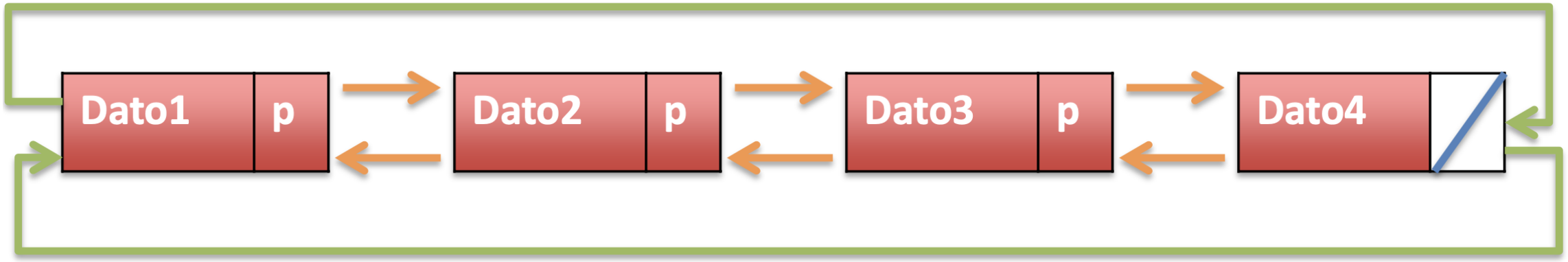
# LISTA CIRCULAR SIMPLEMENTE ENLAZADA

Una lista enlazada simplemente en la que el último elemento (cola) se enlaza al primer elemento (cabeza) de tal modo que la lista puede ser recorrida de modo circular («en anillo»).



# LISTA CIRCULAR DOBLEMENTE ENLAZADA:

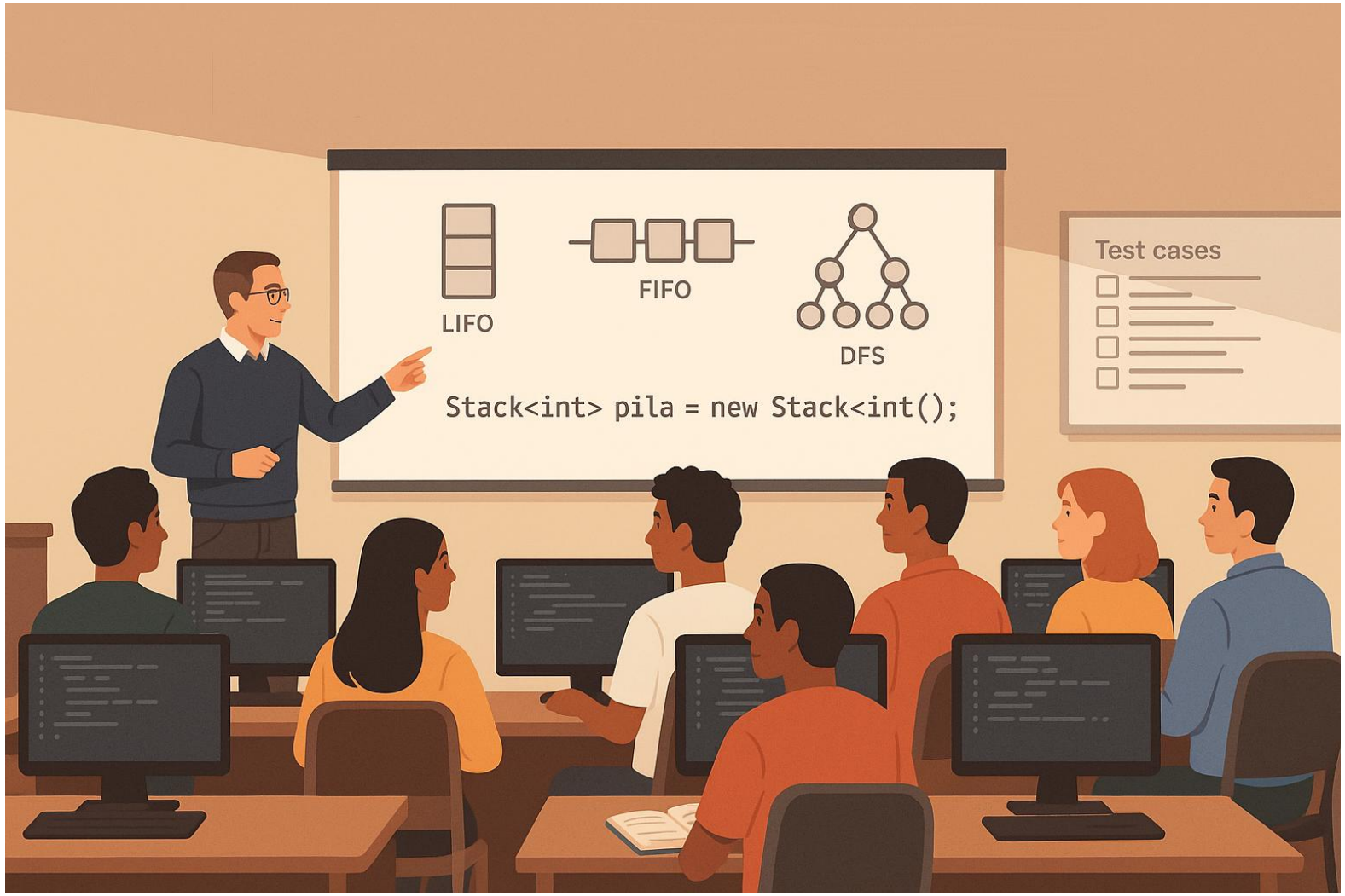
Una lista doblemente enlazada en la que el último elemento se enlaza al primer elemento y viceversa. Esta lista se puede recorrer de modo circular (en anillo) tanto en dirección directa («adelante») como inversa («atrás»).



# OPERACIONES EN LISTAS ENLAZADAS

- Insertar elementos en una lista enlazada
- Mostrar los elementos de una lista enlazada
- Buscar un elemento en una lista enlazada
- Eliminar un elemento en una lista enlazada

# ACTIVIDAD EN CLASE



# PROBLEMA 1

La biblioteca de la universidad "EL ALMA DEL SABER" ha solicitado al área de TI la creación de un sistema para el control de libros por lo cual se requiere tener Codigo (números y letras), Nombre, Autor y Paginas.

Se debe poder realizar las siguientes acciones:

- Insertar por la Izquierda en la lista
- Insertar por la Derecha en la lista
- Listar por izquierdo y derecho

# PROBLEMA 2

Implementar una lista circular que permita ingresar una lista de números.

Debe permitir:

- Insertar
- Borrar
- Mostrar.

# CONCLUSIONES

① La lista doblemente enlazada usa punteros prev y next, permitiendo recorrido bidireccional.

② Insertar y eliminar un nodo conocido cuesta O(1); buscar cuesta O(n).

③ La lista circular simple conecta la cola con la cabeza; siempre necesita condición de parada.

④ La lista circular doble combina ambas capacidades y es la base de LinkedList<T> en .NET.

⑤ En C# implementamos NodoDoble<T> y ListaDoble<T> siguiendo principios SOLID.

# CONCLUSIONES

- La lista enlazada simple constituye el fundamento conceptual y estructural sobre el que se construyen estructuras de mayor complejidad. Las pilas (LIFO) y las colas (FIFO), que se estudiarán en sesiones posteriores, pueden implementarse directamente sobre una lista enlazada simple con operaciones O(1) en sus extremos. Asimismo, las listas de adyacencia para la representación de grafos utilizan este mismo principio. Comprender con precisión el manejo de punteros **Siguiente** en esta sesión es, por tanto, un requisito directo para abordar correctamente dichas estructuras.

- Implementar manualmente una lista enlazada simple a través de las clases **Nodo** y **ListaEnlazada** en C# no es un ejercicio académico aislado, sino el fundamento conceptual que permite comprender cómo se reserva memoria dinámicamente para cada nodo en tiempo de ejecución, cómo los objetos se interconectan mediante referencias, y qué ocurre internamente cuando se inserta o elimina un elemento; comprensión que resulta indispensable para utilizar con criterio y precisión la clase **LinkedList<T>** de la biblioteca estándar de .NET, que implementa una lista doblemente enlazada con operaciones eficientes en ambos extremos, y que un programador sin estos fundamentos utilizaría de forma mecánica, sin capacidad de diagnosticar problemas de rendimiento ni tomar decisiones de diseño justificadas en proyectos reales.

# ¿PREGUNTAS O COMENTARIOS?

# Comprobación del Logro

Cuestionario en la plataforma

| Nivel | Rango |
|---|---|
| Nivel 4 | 17 – 20 |
| Nivel 3 | 13 – 16 |
| Nivel 2 | 9 – 12 |
| Nivel 1 | 0 - 8 |

# Comprobación del Logro

| Criterio | Nivel 4 (Excelente) | Nivel 3 (Bueno) | Nivel 2 (Regular) | Nivel 1 (Deficiente) |
|---|---|---|---|---|
| **Comprensión conceptual** | Demuestra comprensión completa de los conceptos clave. | Comprende la mayoría de los conceptos clave. | Muestra comprensión parcial, con algunas confusiones. | No demuestra comprensión clara de los conceptos. |
| **Aplicación de conocimientos** | Aplica correctamente los conceptos en ejemplos o escenarios. | Aplica conceptos con algunos errores menores. | Aplica de forma superficial o con errores evidentes. | No logra aplicar los conceptos o lo hace incorrectamente. |
| **Claridad y precisión en respuestas** | Las respuestas son claras, bien estructuradas y precisas. | Las respuestas son mayormente claras, con leves imprecisiones. | Respuestas poco claras o con ideas mal organizadas. | Respuestas confusas, incompletas o incoherentes. |
| **Cobertura del contenido esperado** | Responde completamente las 5 preguntas, desarrollando bien cada una. | Responde correctamente 4 de las 5 preguntas. | Responde correctamente 2 o 3 preguntas. | Responde solo 1 pregunta correctamente o no responde. |

# BIBLIOGRAFIA REFERENCIAL


- Ceballos Sierra, F. Microsoft C#: Curso de Programación (2a.ed.) 2014 https://elibronet.eu1.proxy.openathens.net/es/lc/upnorte/titulos/106417
- Cesar Liza Avila; Estructura de datos con C/C++
- Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). Introduction to Algorithms (4th ed.). MIT Press.
- Sedgewick, R., & Wayne, K. (2011). Algorithms (4th ed.). Addison-Wesley Professional.
- Weiss, M. A. (2012). Data Structures and Algorithm Analysis in C# (3rd ed.). Pearson.
- Goodrich, M. T., Tamassia, R., & Goldwasser, M. H. (2013). Data Structures and Algorithms in Java (6th ed.). Wiley.

# GRACIAS

UPN

UNIVERSIDAD
PRIVADA
DEL NORTE